FPGA SYNCHRONOUS-READ FIFO

How the FIFO Works

Signals, memory and data explained step by step for beginners

Module	`fpga_sync_read_fifo` (Verilog-2001)
Target	Xilinx Artix-7 (Arty A7-100T), Vivado 2025.2.1, XSIM
Default size	16 words, 8 bits each (`DEPTH = 16`, `DATA_WIDTH = 8`)
New feature	`data_valid` output flag added to the design and testbench

1. Three Kinds of Signals

Every signal in the design falls into one of three groups.

Group	Signals	Role
Control (you drive them)	`clk`, `rst`, `wr_en`, `rd_en`	Tell the FIFO what you want to happen
Data	`data_in`, `data_out`, `data_valid`	Carry the words in and out
Status (the FIFO drives them)	`full`, `empty`, `count`	Report the FIFO's current state

Two more signals, `wr_accept` and `rd_accept`, are internal. They decide whether a request is actually carried out.

2. The Clock

The `clk` signal is the heartbeat of the design. Almost everything happens **only on the rising edge** of `clk`, inside `always @(posedge clk)`.

Between edges, the registers do not change. At each edge, the FIFO looks at the inputs, decides what to do, and updates its registers all at once.

3. The Memory

```
reg [DATA_WIDTH-1:0] memory [0:DEPTH-1];
```

This reads as 16 boxes, numbered 0 to 15, each holding an 8-bit word.

Address 0	Address 1	Address 2	Address 3	...	Address 15
8-bit word	8-bit word	8-bit word	8-bit word	...	8-bit word

The memory has no logic of its own. The **pointers** decide which box is used.

4. The Pointers

Each pointer holds a **box number** from 0 to 15, so it needs 4 bits (`ADDR_WIDTH = 4`).

- `wr_ptr` is the box where the *next* write will go.
- `rd_ptr` is the box that will be read *next* (the oldest word).

Both start at 0. Each moves forward by 1 after its operation. After box 15 it wraps back to 0:

`0 → 1 → 2 → ... → 14 → 15 → 0 → 1 → ...`

This is why it is called a **circular buffer**. Old boxes are reused once their data has been read.

5. Why a Counter as Well?

Both pointers can show the same number when the FIFO is empty and also when it is full. For example, if both are at box 5, you cannot tell which case it is. The `count` register settles that:

```
assign full  = (count == DEPTH);   // 16 words stored
assign empty = (count == 0);       // 0 words stored
```

The `assign` statements are **continuous**. `full` and `empty` update immediately whenever `count` changes, with no clock needed.

6. Requests versus Actions

```
wire wr_accept = wr_en && !full;
wire rd_accept = rd_en && !empty;
```

`wr_en` and `rd_en` are only **requests**. The FIFO acts on them only when it is safe:

wr_en	full	wr_accept	Result
1	0	1	Write happens
1	1	0	Write ignored (no overflow)
0	x	0	Nothing

Reads work the same way, using `rd_en` and `empty`.

7. What Happens on Each Clock Edge

At every rising edge the FIFO does the following, in order of priority:

1. **If `rst` = 1:** clear everything (`wr_ptr`, `rd_ptr`, `data_out`, `data_valid`, `count` all become 0). Nothing else happens.
2. **Otherwise, if `wr_accept`:** store `data_in` in `memory[wr_ptr]`, then advance `wr_ptr`.
3. **Set `data_valid`** to the value of `rd_accept`.
4. **If `rd_accept`:** copy `memory[rd_ptr]` into `data_out`, then advance `rd_ptr`.
5. **Update `count`:** +1 for a write only, −1 for a read only, no change otherwise.

The write path and the read path are independent. That is why they can happen **in the same clock cycle**.

8. The <= Operator

The code uses `<=`, called a **nonblocking assignment**. Its rule is that **all right-hand sides are read first, and all left-hand sides are updated together at the end of the edge**.

This has two consequences:

- During an edge, every signal still shows its **old** value. So `full`, `empty` and `count` are the values from *before* the edge.
- The new values appear **just after** the edge.

> **Why it matters:** `data_out` shows a read word one edge later. At the edge it is loaded, and you see it right after.

9. A Cycle-by-Cycle Trace

Start from reset. The table shows values **after** each rising edge.

Edge	Inputs	What happens	wr_ptr	rd_ptr	count	data_out	data_valid
reset	`rst=1`	Everything cleared	0	0	0	00	0
1	`wr_en=1`, `data_in=11`	`memory[0] = 11`	1	0	1	00	0
2	`wr_en=1`, `data_in=22`	`memory[1] = 22`	2	0	2	00	0
3	`wr_en=1`, `data_in=33`	`memory[2] = 33`	3	0	3	00	0
4	`rd_en=1`	`data_out = memory[0]`	3	1	2	**11**	**1**
5	`rd_en=1`, `wr_en=1`, `data_in=44`	Read `memory[1]`, write `memory[3] = 44`	4	2	2	**22**	**1**
6	nothing	Idle	4	2	2	22 (held)	**0**

Things to notice:

- **Edge 4:** `data_valid` rises on the same edge that `data_out` gets its new word.
- **Edge 5:** a read and a write occur together, so the count stays at 2.
- **Edge 6:** with no read, `data_out` keeps its old value but `data_valid` drops. That is how a consumer tells fresh data from stale data.

10. A Wrap-Around Example

Suppose `wr_ptr` is 15 and a write is accepted:

```
if (wr_ptr == DEPTH-1)  wr_ptr <= 0;   // 15 -> 0
else                    wr_ptr <= wr_ptr + 1;
```

The word goes into `memory[15]`, and the next write goes into `memory[0]`. This is safe because `count` guarantees you never overwrite unread data. A write to a full FIFO is rejected first.

11. The Testbench and These Signals

The testbench plays the role of the outside world:

- It generates `clk` and drives `rst`, `wr_en`, `rd_en` and `data_in`.
- It watches `data_out`, `data_valid`, `full`, `empty` and `count`.
- It changes inputs on the **falling** edge, so they are stable when the FIFO samples on the **rising** edge.
- It waits `#1` after the rising edge, so the new values have appeared before it checks them.

12. Summary

Item	Purpose
`memory`	Holds the data words
`wr_ptr`	Next box to write
`rd_ptr`	Next box to read
`count`	How many words are stored
`full`, `empty`	Derived from `count`, used to block unsafe operations
`wr_accept`, `rd_accept`	Request AND safe
`data_out`	The word read, updated on the clock edge
`data_valid`	Says whether `data_out` is fresh

> **In one sentence:** The FIFO is a **circular buffer** (memory plus two pointers) with a **counter** that gives the full and empty flags. Reads and writes are accepted only when safe, everything is clocked, and `data_valid` tells you when `data_out` holds a freshly read word.